

Robotic Arm - CDC

(CDC: Curriculum Design Collaboration)

Final Evaluation Report

University of Westminster

Student Partners:

Charanjot Sidhu, Caleb Suleaudu, Suber Abdi, Nicholas Nicolaou

Staff Partner:

Muharrem Maloku

1. Abstract

For this project we designed, built and customised an interactive robot arm, with the long-term idea being that it could be used as a teaching tool. Instead of being a finished, untouchable product, the arm was meant to be something students could take apart, put back together, or build on top of, so they could actually see how it works rather than just being told.

We started with a buildable robot arm kit and modified it to suit what we wanted it to do, rather than designing and manufacturing everything from nothing. The goal was for the arm to be controlled by a joystick and to play noughts and crosses using a camera to track moves. It runs on stepper motors, a Raspberry Pi and Arduino boards for the joystick side of things.

Building it went roughly in this order: the arm itself, then the boards, then the motors, then wiring it all together. After that came the coding - first getting the Raspberry Pi to calibrate and move the motors in Python, then the joystick code (which meant getting the Arduino and the Pi talking to each other), and finally the camera.

Unfortunately we ran into motor and battery problems near the end that we couldn't fully get to the bottom of in the time we had, on top of finding it hard to get the whole group together as often as we needed due to other coursework. So the arm never quite got to the point of reliably playing noughts and crosses. That said, we still came away from it having learned a lot across software, hardware and mechanical work, and we think the idea itself - a robot arm kit that can be pulled apart and rebuilt - still holds up as something genuinely useful for teaching.

2. Background and Aims

Background

The project came about because we wanted to build something that combined hardware, software and mechanical work into one build, rather than doing these things separately as is usually the case on a course. We also wanted it to have a use beyond just being a demonstration - the plan was for it to work as an educational kit that could be dismantled and rebuilt by other students, so they could learn from how it was actually put together rather than just seeing a finished product. This felt like a gap worth filling, since a lot of learning in this area tends to happen in separate silos rather than as one combined, hands-on project.

Aims

The aim was simple to state even if it wasn't simple to do: build a working, customisable robot arm that could play noughts and crosses against a person, using a joystick controller and a camera to track the game.

Objectives

- Assemble a buildable robot arm kit and adapt it to what we needed
- Get the stepper motors moving the arm as intended
- Write the Raspberry Pi code to calibrate and control the motors
- Write the Arduino code for the joystick controls
- Add a camera to track and recognise moves during gameplay

Impact on Learning and Teaching

This project pushed us to learn things we wouldn't necessarily pick up in lectures, since coding a physical system that has to actually move and respond is quite different to writing code for an assignment. Between the four of us we ended up covering software, hardware and mechanical work, so it stretched us across areas we don't normally get to combine like this.

3. Methods

We bought a buildable robot arm kit as our starting point and then adjusted and customised it to fit what we were trying to achieve, rather than designing and manufacturing an arm from nothing. Assembly went section by section - the arm structure first, then the boards, then the motors, then connecting everything together.

Once it was physically assembled we moved onto the coding side. This started with Python code for the Raspberry Pi, to set the motor calibrations and get the board actually driving the motors. Next was the joystick code, which needed the Arduino and the Pi to communicate with each other properly. The camera came last, both in terms of the code and physically mounting it onto the arm.

It was just the four of us and our supervisor, Muharrem Maloku, involved - no outside participants, so there was no need for a Participant Information Sheet. Safety still mattered though, mainly around making sure everything was wired correctly and securely, and that the arm was switched off properly before making adjustments or connecting the batteries, so we didn't risk it short-circuiting.

We tested things in stages rather than all at once. For the joystick, we used the Arduino's onboard LED to check it was actually registering input before connecting it to anything else. The motors were tested running off USB power before we moved to batteries, and the batteries themselves were checked on separate circuits with a voltmeter before going anywhere near the arm.

4. Results

By the end, the arm was close to being finished but not quite there. The biggest issue was the motors - they had worked fine before, then stopped functioning properly, and we

weren't able to get them working again in time. If they had kept working, the arm would have been able to actually play noughts and crosses as planned.

Along the way the arm had to be taken apart and reassembled more than once, and the Raspberry Pi needed resetting and recoding a few times before it behaved. Battery life turned out to be a bigger problem than expected too - the rechargeable batteries drained much faster than we thought they would, and when we tried replacing them, the new ones weren't actually compatible with the arm. Between the motor fault, the battery issues, and simply not having enough time to meet as a group as much as we needed, we couldn't pin down whether the final problem was mechanical, electrical, or something in the code.

5. Discussion

The main things that held us back were the motor fault, the battery problems, and just not being able to get everyone together as often as we'd planned because of coursework elsewhere. Even so, we don't think the approach itself was wrong - starting from a buildable kit and customising it rather than building completely from scratch still feels like the right call, since what went wrong was mostly outside our control rather than a flaw in how we went about it.

Most of our results matched what we expected, apart from the motors and batteries failing right near the end. Both had been working the day before, then just stopped, and we couldn't work out why without either taking the whole arm apart to check the wiring or going back through every line of code. That was probably the most frustrating part of the whole project - not knowing if it was a hardware fault or something we'd coded wrong.

If we had finished it, we think this could genuinely be useful in a teaching setting. A robot arm that students can dismantle and rebuild themselves gives a way into learning robotics and coding that you don't really get from a textbook, and it would let people engage with hardware, software and mechanical design all through one project rather than treating them separately.

On the group side, having four of us working on different bits at once meant harder problems usually had a few different people looking at them, which helped. The main struggle was timing - it got harder as the term went on to get everyone in the same place at the same time. We got around this by splitting up tasks to do at home, or having whoever couldn't be there in person join on a video call instead.

6. Conclusions and Future Work

If there's one thing this project drove home, it's that time management matters more than we gave it credit for at the start, and that the hardware side of a build like this takes a lot longer than you'd expect going in.

For anyone picking this up in the future, the first job would be figuring out exactly what's wrong with the motors - whether they need retuning, the wiring's damaged, or it's actually a code issue. Getting hold of batteries that are actually compatible with the arm would be the other priority, since our replacements weren't right. Once those are sorted, the last piece would be finishing off the joystick and camera integration so the arm can reliably play a full game, which was the original goal.

7. Lessons Learned

The biggest lesson for us was around time management - both in terms of getting the group together and in underestimating just how long the hardware side of things would take. We also learned that it pays to make decisions early rather than going back and forth; for example, we spent longer than we should have deciding whether to start from a kit or build the arm ourselves from the ground up, and that early indecision ended up costing us time later on.

8. Group Reflection

One of the things that worked well was having several of us working on different parts of the build at the same time - it meant that when something tricky came up, there were a few different people who could look at it from a different angle. Splitting tasks up and letting each person go deep on their own part (software, hardware or mechanical) was a big part of why we got as far as we did.

Where we struggled most was scheduling - as the project went on it got harder to get everyone together at once, and on top of that we kept having to take the arm apart and put it back together to fix smaller issues. We dealt with the scheduling side by giving people tasks to do independently at home, or bringing them in over video call when they couldn't make it in person.

Looking back, we'd put more thought into scheduling from the start next time, and we'd try to make bigger decisions - like whether to buy a kit or build from scratch - earlier rather than going back and forth. None of these were huge issues on their own, but a few smaller ones like this added up over the course of the project.